Semester 2 Lab 9: Introduction to Android Development with Android Studio

Introduction

Welcome to your first Android development lab! In this session, we will take our first steps towards building Android applications. Android apps are developed using Android Studio, the official Integrated Development Environment (IDE) for Android development. Android Studio is similar to other IDEs you may have used, such as Spring Tool Suite, but it is specifically tailored for Android development. By the end of this lab, you will have created your first Android app and learned the basics of Android Studio.

What is Android Studio?

Android Studio is a powerful IDE designed by Google for building Android apps. It provides tools for coding, debugging, testing, and deploying Android applications. It also includes a visual layout editor, an emulator for testing apps, and support for various programming languages like Java and Kotlin.

By the end of this lab, you will have created a simple Android app and learned the basic components involved in Android app development.

Setup

Before we start coding, let's ensure that Android Studio is set up correctly. Android Studio is already installed on the lab computers. If you want to install it on your personal computer, you can download it from the official website: Android Studio Download. Make sure to use the latest version for the best experience.

Performance Tips

Android Studio is a resource-intensive application, which means it can slow down your computer if you don't have enough memory or processing power. Here are some tips to improve performance:

1. **Use a Physical Android Device**: If possible, connect an Android device to your computer via USB. This will give you a **big performance boost** compared to using an emulator.
2. **Close Unnecessary Applications**: Android Studio works best when it has access to most of your computer's resources. Close any unnecessary applications, especially memory-hungry ones like web browsers with many tabs open.
3. **Reduce Android Studio's Memory Usage**: If your computer has 4GB of RAM or less, you may need to reduce Android Studio's memory usage. To do this:
    - Go to Help > Edit Custom VM Options....

- Replace the default -Xmx2048m (2GB) with a smaller value, such as -Xmx1024m (1GB) or -Xmx512m (512MB). This will make Android Studio slower but will prevent your system from freezing.

4. **Enable Power Save Mode**: You can reduce Android Studio's resource usage by enabling Power Save Mode. Go to File > Power Save Mode. This will disable background operations like code style checks and code completion, but it will help conserve memory.
5. **Disable Antivirus for Your Project Directory**: Some antivirus programs may slow down Android Studio by scanning your project files every time you recompile your code. You can temporarily disable your antivirus for your project directory to improve performance.
6. **Keep Gradle Updated**: Gradle is the build system used by Android Studio. Make sure to install any Gradle updates that Android Studio prompts you to install.

Creating Your First Android Project

Now that we have Android Studio set up, let's create our first Android project.

Step 1: Start a New Project

1. Open Android Studio. You will see the Welcome Screen. Click on New Project to start a new project.

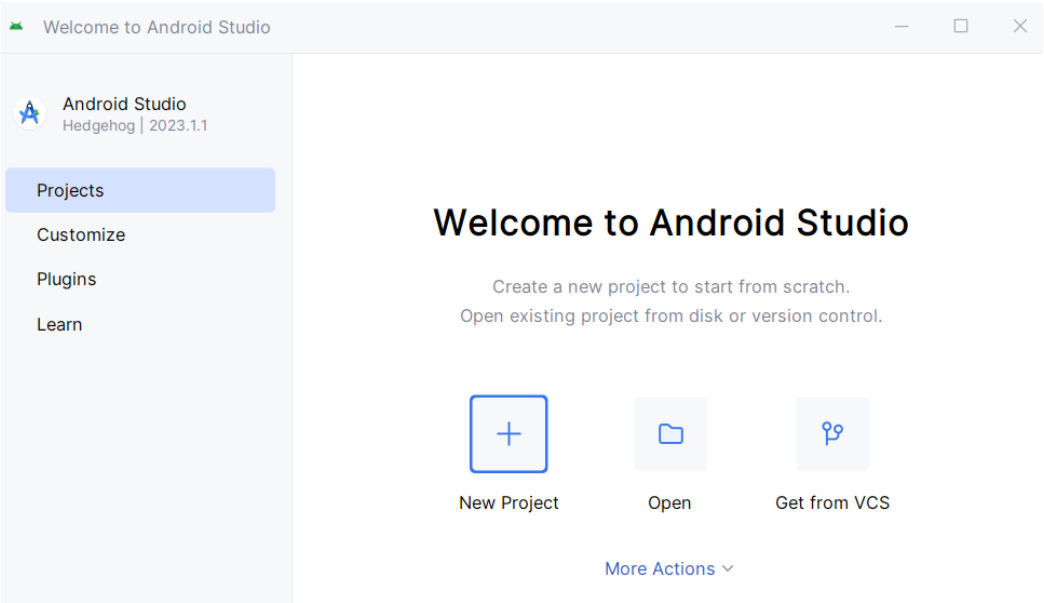


Figure 1: Android Studio Welcome Screen

2. In the "Create New Project" wizard:
    - Select Basic Views Activity for the template under Phone and Tablet.
    - Click Next.

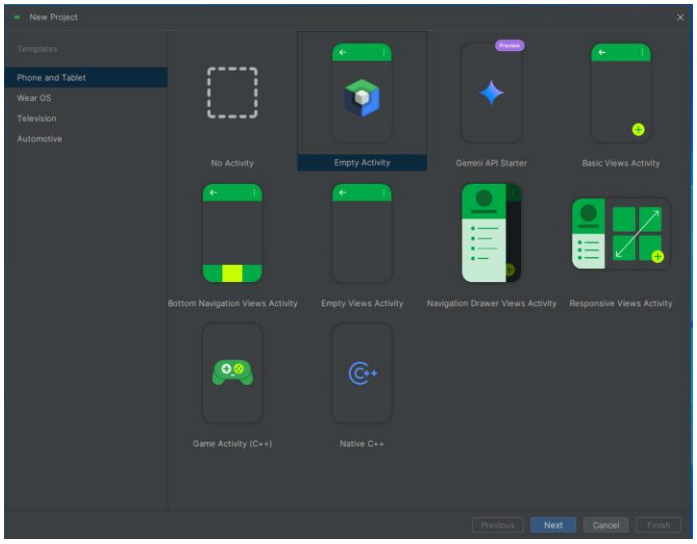


Figure 2: Android New Project Template

Step 2: Configure Your Project

On the Configure Your Project screen, fill out the following details:

- Project Name: CO2124 First App
- Package Name: uk.edu.le.co2124firstapp
- Language: Select Java (this will be the language used for your app, but feel free to use other languages in future projects).
- Minimum SDK: Choose API 26: Android 8.0 (Oreo). This ensures your app will work on most devices (95.4% of Android devices). Observe that we also need to decide the Minimum SDK. In this field, we are indicating which version of Android our app will run on. For example, by selecting API 24: Android 7.0 (Nougat), we are showing that our app will run on any device which runs Android 7.0 or later. Android Studio provides us with information about the proportion of existing devices on which our app will run. In our case, it is 95.4% of all Android devices. If you were to choose Android 4.0 (Jelly Bean), you would be supporting all existing Android devices (at the cost of not having access to lots of features and APIs).

Your configuration screen should look like this:

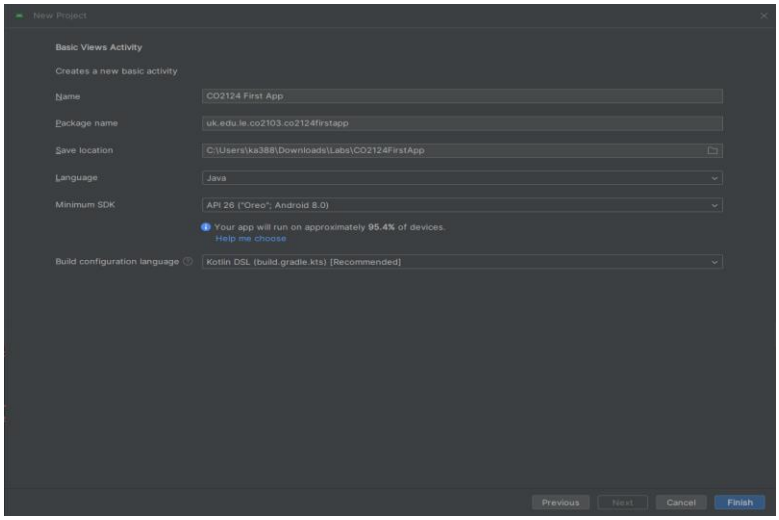


Figure 3: New Project Configuration

Click **Finish** to create the project.

Exploring the Android Studio Interface

Once your project is created, you will see the main Android Studio interface:

- On the left, there is the **Project Explorer**, which shows all the files and folders for your app.
- In the centre is the **Code Editor**, where you will write your Java code.
- To the right, you will see various panels and tools, including the **Layout Editor** and **Attributes** panels.

The following key files will be present in your project:

- MainActivity.java (located in **app > java > uk.edu.le.co2124firstapp**): This is the main entry point for your app. When you build and run your app, the system launches an instance of this Activity and loads its layout.
- activity_main.xml (located in **app > res > layout**): This file defines the user interface (UI) of your app.
- AndroidManifest.xml (located in **app > manifests**): This file declares app components like MainActivity and other settings. including the Activity that will execute when the app starts, i.e., MainActivity with the **intent** android.intent.action.MAIN:

```
<application
    ...
    <activity
            android:name=".MainActivity"
      android:exported="true"
      android:theme="@style/Theme.CO2124FirstApp">
```

```
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    </application>
```

- build.gradle (found under **Gradle Scripts**): This file handles project configuration and dependencies.

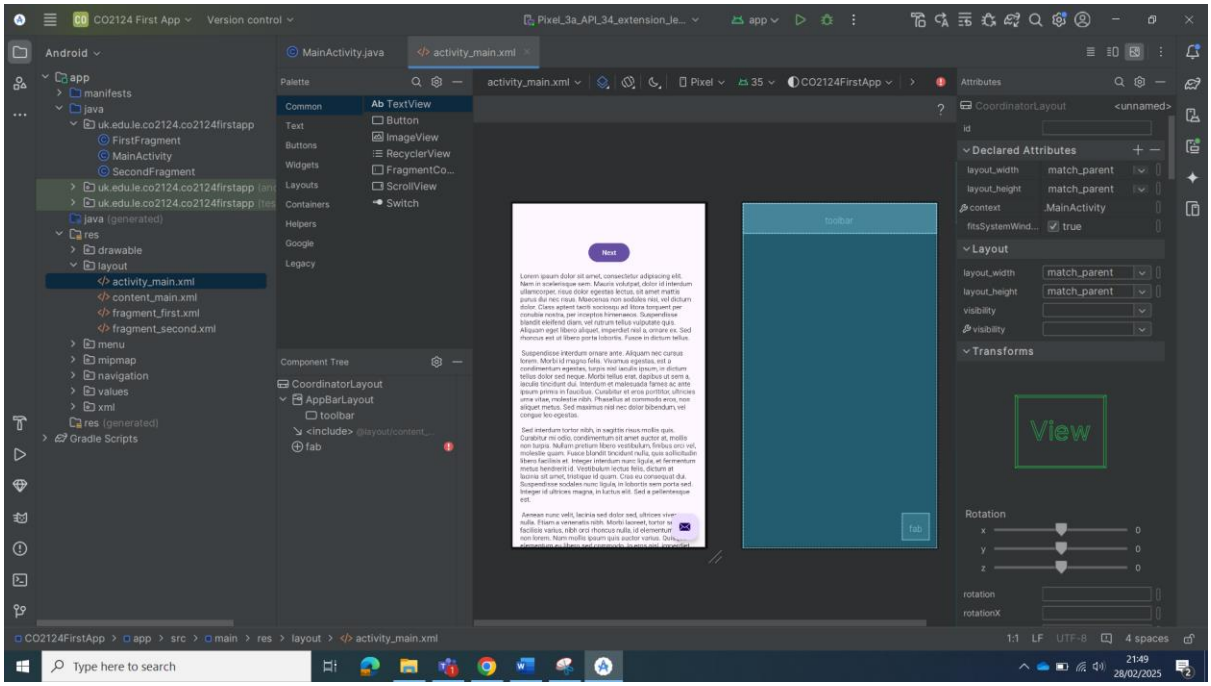


Figure 4: Android Design View

Running your Project

There are two ways to run your app: using a real Android device or an emulator. In this lab, we will use an emulator.

Setting Up an Emulator:

1. You will need an Android Virtual Device (AVD) for the emulator. This might take some time, as you will have to download some large files. Follow these instructions to set up one:
    - Go to Tools > AVD Manager.
    - Create a new AVD using the Pixel 6 device and API Level 34.
2. Once the AVD is set up, go to the toolbar and select your app from the run configurations dropdown.

3. Click the **Run** button (the green play icon) to install and launch your app on the emulator. Android Studio will now install your app on the AVD and start the emulator. You now should see your app displayed in the emulator:

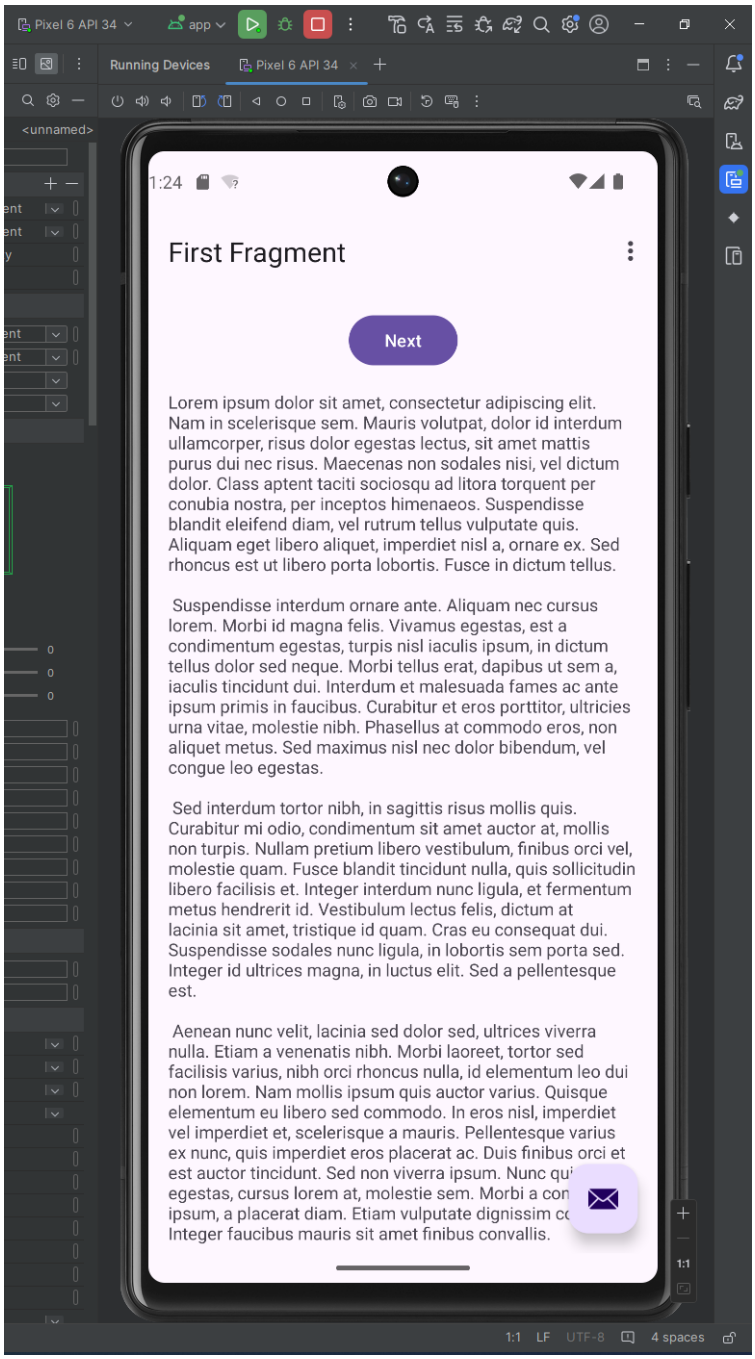


Figure 5: Android Emulator

Interact with your application. What happens if you click on NEXT? What happens if you click on the floating button with the envelope icon?

The above are two sample actions that Android Studio has implemented for you. To look at the code that implements it, go back to Android Studio and open MainActivity.java. Inspect the onCreate method to see how the floating button action is created:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);

  binding = ActivityMainBinding.inflate(getLayoutInflater());
  setContentView(binding.getRoot());

  setSupportActionBar(binding.toolbar);

  NavController navController = Navigation.findNavController(this,
R.id.nav_host_fragment_content_main);
  appBarConfiguration = new AppBarConfiguration.Builder(navController.getGraph()).build();
  NavigationUI.setupActionBarWithNavController(this, navController, appBarConfiguration);

  binding.fab.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
      Snackbar.make(view, "Replace with your own action", Snackbar.LENGTH_LONG)
          .setAnchorView(R.id.fab)
          .setAction("Action", null).show();
    }
  });
}
```

Editing the UI

Now that you have your app running, let's add more functionality to it by editing the UI.

Step 1: Modify the Layout

1. Double-click on activity_main.xml in the Project Explorer to open the Layout Editor. If you are viewing the XML code, click on Design in the top-right corner (see highlighted area below). From the Design Surface drop-down menu, choose Blueprint. All of these are shown in the red boxes in the figure below

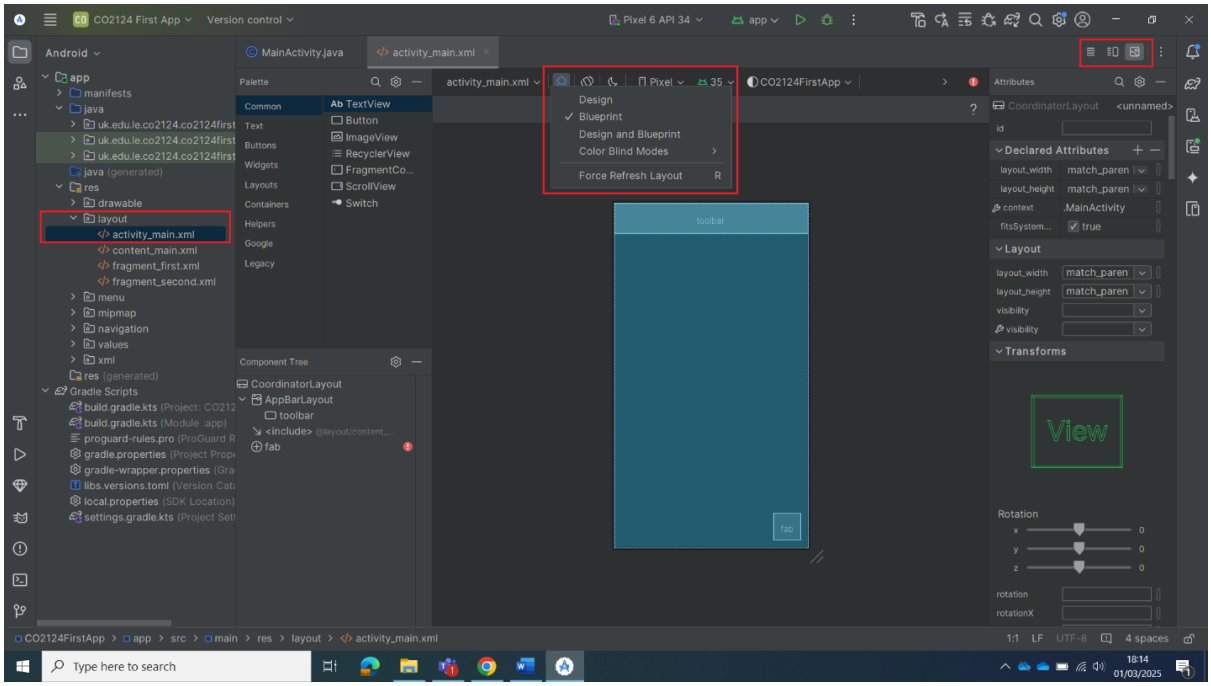


Figure 6: Android Blueprint View

Step 2: Add a Text Field and Button

We will now add a Text Field and a Button to the first fragment of our app.

1. Open fragment_first.xml in the layout editor.
2. Delete the existing TextView (the one with the "Lorem Ipsum" text).
3. From the Palette panel on the left, drag and drop a Plain Text field into the layout. Set its id to editTextName.
4. Drag and drop a Button from the Palette and set its id to buttonSend.

Step 3: Define Constraints

The layout uses a ConstraintLayout, which allows you to position UI elements relative to each other. To define constraints:

1. Click on the Text Field and drag the circles on its sides to connect it to the top and sides of the screen.
2. Connect the Button to the bottom of the Text Field and the sides of the screen.
3. If you see this error - editTextName <EditText>: Touch target size too small android studio, change android:layout_height="48dp" in your XML code.

Your layout should now look something like this:

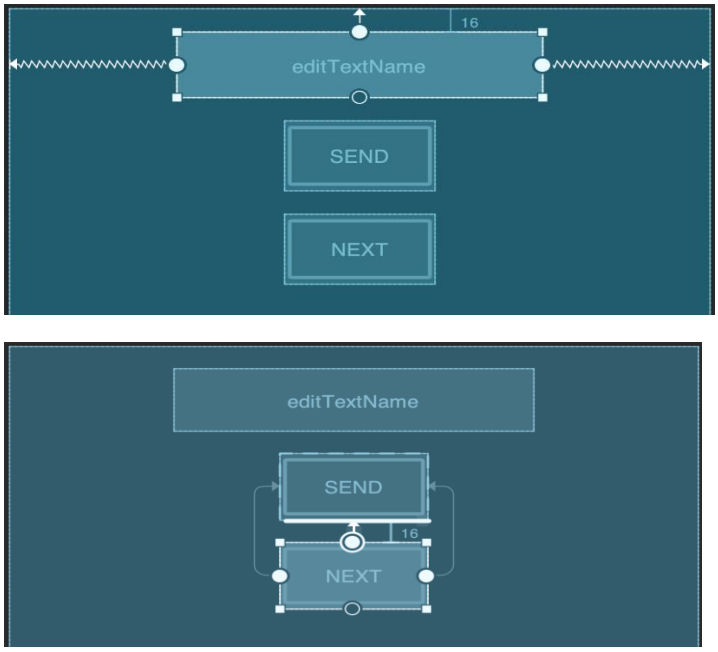
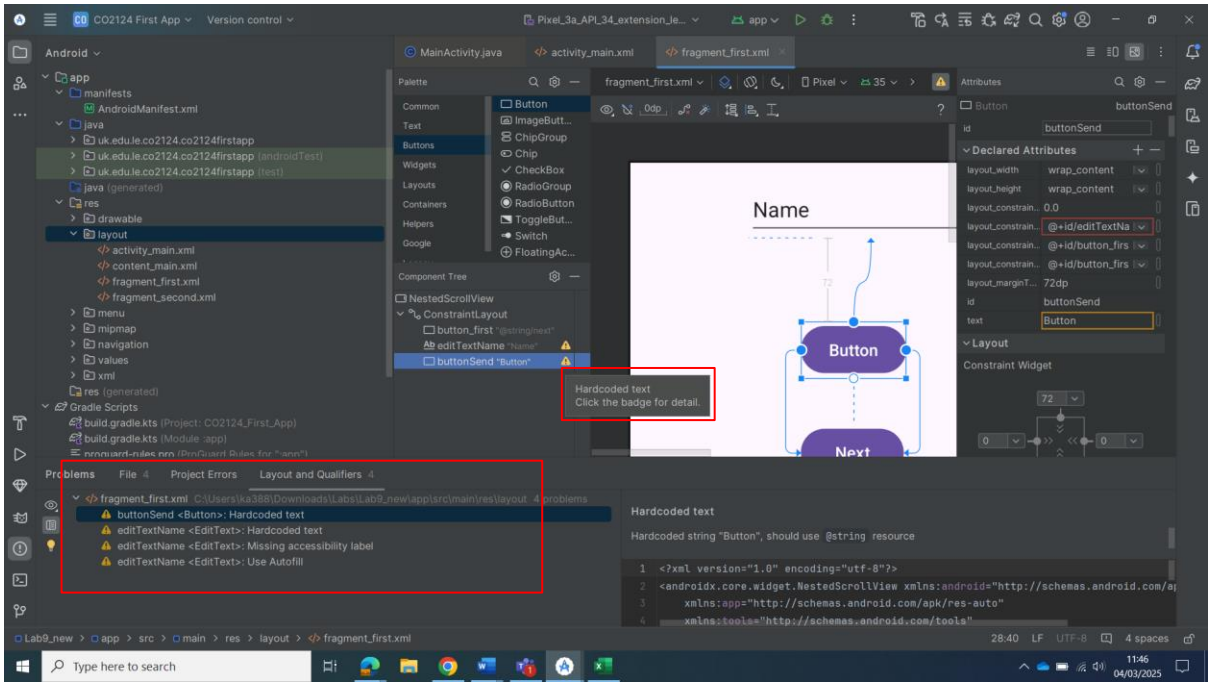


Figure 7: Positioning of Views Relative to Each Other

Add String Resources

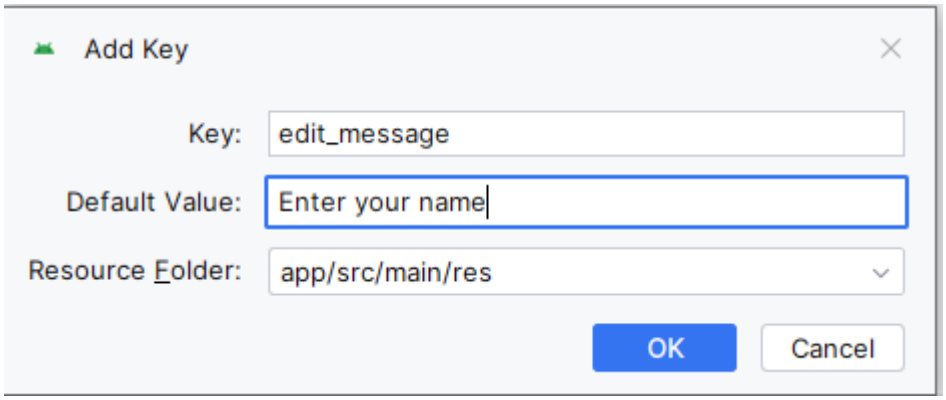
Android Studio will warn you that the text in your views is hardcoded. To fix this, we will use string resources.



1. Open strings.xml under app > res > values.
2. You can edit the XML file directly or click on **Open editor** at the top of the window to open the **Translations Editor**, which provides a simple interface to add and edit your default strings, including translations.

3. Add the following strings:
    - edit_message: "Enter your name"
    - button_send: "Send"

Now, go back to fragment_first.xml and set the hint property of the Text Field to @string/edit_message and the text property of the Button to @string/button_send.



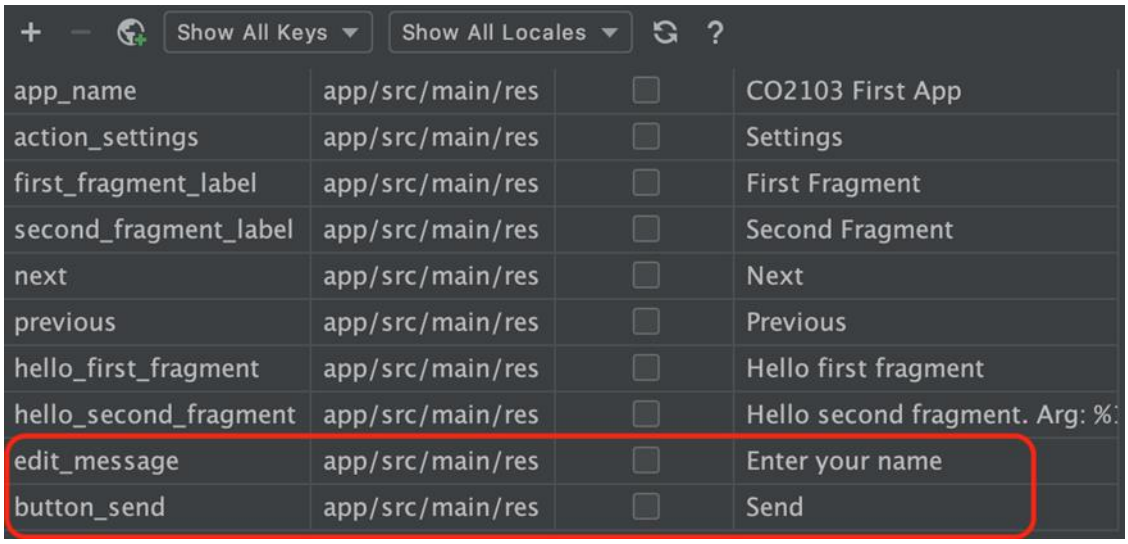


Figure 8: Translations Editor

If you now run your app, your final fragment layout should look like this (notice that for the text field, we have set a placeholder, or hint, instead of the actual text):

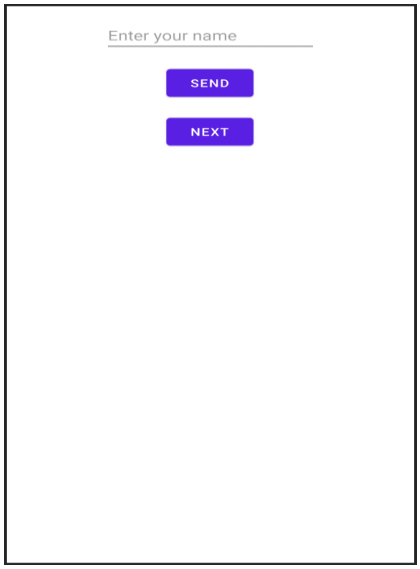


Figure 9: Final Fragment Layout

Responding to Button Clicks

Now, let's make the **Send Button** functional.

1. Open FirstFragment.java and add the following code to handle button clicks:

add the following global constant:

```
public class FirstFragment extends Fragment {

    public static final String USER_NAME = "uk.edu.le.co2124.firstapp.USERNAME";
    ...
}
```

In the same class, you will add some code to the onViewCreated method:

```
// Add functionality for the Send button
Button button = (Button) view.findViewById(R.id.buttonSend);
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        Intent intent = new Intent(getActivity(), GreetingActivity.class);
        EditText editText = (EditText) view.getRootView().findViewById(R.id.editTextName);
        String message = editText.getText().toString();
        intent.putExtra(USER_NAME, message);
        startActivity(intent);
    }
});
```

2. This code listens for clicks on the button and sends the message from the EditText to a new activity called GreetingActivity. This code creates an **Intent** that will navigate to a new Activity called GreetingActivity when the button is clicked. It also passes the text entered in the **Text Field** to the new Activity.

Create the GreetingActivity

1. Right-click on the app directory in the Project Explorer and select New > Activity > Empty Activity.
2. Rename the activity to GreetingActivity.
3. In GreetingActivity XML, add a TextView to display the greeting message. Set its id to textView and center it on the screen.
4. In GreetingActivity.java, add code to the onCreate method to display the message passed from the FirstFragment

```
setContentView(R.layout.activity_greeting);

Intent intent = getIntent();
String message = intent.getStringExtra(FirstFragment.USER_NAME);

TextView textView = findViewById(R.id.textView);
textView.setText("Hi, " + message);
```

What you are doing here is receiving the string USER_NAME from the main Activity, and you are writing it into your text view.

Final Steps: Running the App

Once you've completed these steps, click the **Apply Changes** button (the icon looks like a lightning bolt) to apply your changes.

Your app should now allow users to input their name, click **Send**, and see a greeting message in a new activity.

Next steps

Once you have completed this worksheet, feel free to explore Android Studio and the different features it offers you. In the next lab, we will be implementing another app with a richer set of features.